

Python for Data Science Interview Questions & Answers Cheat Sheet

Table of Contents

1. [Python Fundamentals for Data Science](#)
2. [NumPy Interview Questions](#)
3. [Pandas Interview Questions](#)
4. [Data Manipulation & Cleaning](#)
5. [Data Visualization \(Matplotlib/Seaborn\)](#)
6. [Scikit-Learn & Machine Learning](#)
7. [Performance Optimization](#)
8. [Data Structures & Algorithms](#)
9. [File I/O & Data Formats](#)
10. [Memory Management](#)
11. [Error Handling & Debugging](#)
12. [Advanced Python Concepts](#)

Python Fundamentals for Data Science

Q1: What makes Python suitable for data science, and what are its key advantages?

Answer:

Key Python Advantages for Data Science:

Advantage	Description	Example
Rich Ecosystem	Extensive libraries for data science	NumPy, Pandas, Matplotlib, Scikit-learn
Readable Syntax	Easy to learn and maintain	Clear, English-like syntax
Interpreted Language	Interactive development and testing	Jupyter notebooks, IPython
Cross-platform	Runs on multiple operating systems	Windows, Mac, Linux
Large Community	Extensive documentation and support	Stack Overflow, GitHub
Integration	Easy integration with other tools	R, SQL, Spark, cloud services

Python Data Science Ecosystem:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

class DataScienceWorkflow:
    def __init__(self):
        self.data = None
        self.model = None

    def load_and_explore_data(self, file_path):
        """Comprehensive data loading and exploration"""

        # Load data
        self.data = pd.read_csv(file_path)

        # Basic exploration
        exploration_results = {
            'shape': self.data.shape,
            'columns': list(self.data.columns),
            'dtypes': self.data.dtypes.to_dict(),
            'missing_values': self.data.isnull().sum().to_dict(),
            'summary_stats': self.data.describe().to_dict(),
            'memory_usage': self.data.memory_usage(deep=True).sum()
        }

        # Quick visualization
        fig, axes = plt.subplots(2, 2, figsize=(15, 10))

        # Missing values heatmap
        sns.heatmap(self.data.isnull(), ax=axes[0,0], cbar=True, cmap='viridis')
        axes[0,0].set_title('Missing Values Pattern')

        # Correlation heatmap (for numeric columns)
        numeric_cols = self.data.select_dtypes(include=[np.number])
        if len(numeric_cols.columns) > 1:
            sns.heatmap(numeric_cols.corr(), annot=True, ax=axes[0,1], cmap='coolwarm')
            axes[0,1].set_title('Correlation Matrix')

        # Data types distribution
        dtype_counts = self.data.dtypes.value_counts()
        axes[1,0].pie(dtype_counts.values, labels=dtype_counts.index, autopct='%1.1f%%')
        axes[1,0].set_title('Data Types Distribution')

        # Sample data preview
        axes[1,1].axis('off')
        table_data = self.data.head().values
        axes[1,1].table(cellText=table_data,
                        colLabels=self.data.columns,
                        cellLoc='center', loc='center')
        axes[1,1].set_title('Data Preview (First 5 Rows)')

        plt.tight_layout()

```

```

plt.show()

return exploration_results

def preprocessing_pipeline(self, target_column):
    """Complete preprocessing pipeline"""

    # Separate features and target
    X = self.data.drop(columns=[target_column])
    y = self.data[target_column]

    # Handle missing values
    numeric_columns = X.select_dtypes(include=[np.number]).columns
    categorical_columns = X.select_dtypes(include=['object', 'category']).columns

    # Fill numeric missing values with median
    X[numeric_columns] = X[numeric_columns].fillna(X[numeric_columns].median())

    # Fill categorical missing values with mode
    for col in categorical_columns:
        X[col] = X[col].fillna(X[col].mode().iloc[0] if len(X[col].mode()) > 0 else None)

    # Encode categorical variables
    X_encoded = pd.get_dummies(X, columns=categorical_columns, drop_first=True)

    return X_encoded, y

def train_and_evaluate_model(self, X, y, test_size=0.2, random_state=42):
    """Train and evaluate a machine learning model"""

    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Train model
    self.model = RandomForestClassifier(n_estimators=100, random_state=random_state)
    self.model.fit(X_train, y_train)

    # Make predictions
    y_pred_train = self.model.predict(X_train)
    y_pred_test = self.model.predict(X_test)

    # Evaluate performance
    train_accuracy = accuracy_score(y_train, y_pred_train)
    test_accuracy = accuracy_score(y_test, y_pred_test)

    # Feature importance
    feature_importance = pd.DataFrame({
        'feature': X.columns,
        'importance': self.model.feature_importances_
    }).sort_values('importance', ascending=False)

    results = {
        'train_accuracy': train_accuracy,
        'test_accuracy': test_accuracy,

```

```

        'feature_importance': feature_importance,
        'model': self.model
    }

    print(f"Training Accuracy: {train_accuracy:.4f}")
    print(f"Testing Accuracy: {test_accuracy:.4f}")
    print(f"Overfitting Check: {train_accuracy - test_accuracy:.4f}")

    return results

# Example usage demonstration
def demonstrate_python_for_data_science():
    # Create sample dataset
    np.random.seed(42)
    n_samples = 1000

    data = {
        'age': np.random.normal(35, 12, n_samples),
        'income': np.random.lognormal(10.5, 0.8, n_samples),
        'education': np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'], n_samples),
        'experience': np.random.exponential(5, n_samples),
        'target': np.random.choice([0, 1], n_samples, p=[0.6, 0.4])
    }

    df = pd.DataFrame(data)
    df.to_csv('sample_data.csv', index=False)

    # Run workflow
    workflow = DataScienceWorkflow()
    exploration = workflow.load_and_explore_data('sample_data.csv')

    X_processed, y = workflow.preprocessing_pipeline('target')
    model_results = workflow.train_and_evaluate_model(X_processed, y)

    return workflow, exploration, model_results

# Uncomment to run demonstration
# workflow, exploration, results = demonstrate_python_for_data_science()

```

Q2: Explain Python's data types and their relevance in data science.

Answer:

Python Data Types for Data Science:

```

import numpy as np
import pandas as pd
from collections import defaultdict, Counter, OrderedDict
from datetime import datetime, timedelta
import sys

class DataTypesDemo:
    def __init__(self):
        self.examples = {}

```

```

def demonstrate_basic_types(self):
    """Demonstrate basic Python data types"""

    # Numeric Types
    integer_val = 42 # int
    float_val = 3.14159 # float
    complex_val = 3 + 4j # complex

    # Text Type
    string_val = "Data Science" # str

    # Boolean Type
    boolean_val = True # bool

    # Sequence Types
    list_val = [1, 2, 3, "mixed", True] # list - mutable
    tuple_val = (1, 2, 3, "immutable") # tuple - immutable
    range_val = range(0, 10, 2) # range

    # Mapping Type
    dict_val = {"name": "Alice", "age": 30, "skills": ["Python", "ML"]}

    # Set Types
    set_val = {1, 2, 3, 4, 5} # set - mutable, unique elements
    frozenset_val = frozenset([1, 2, 3]) # frozenset - immutable

    self.examples['basic_types'] = {
        'numeric': [integer_val, float_val, complex_val],
        'text': string_val,
        'boolean': boolean_val,
        'sequence': [list_val, tuple_val, range_val],
        'mapping': dict_val,
        'sets': [set_val, frozenset_val]
    }

    return self.examples['basic_types']

def demonstrate_data_science_types(self):
    """Demonstrate NumPy and Pandas data types"""

    # NumPy data types
    numpy_types = {
        'int8': np.int8(127), # 1 byte integer
        'int16': np.int16(32767), # 2 byte integer
        'int32': np.int32(2147483647), # 4 byte integer
        'int64': np.int64(9223372036854775807), # 8 byte integer
        'float16': np.float16(3.14), # Half precision
        'float32': np.float32(3.14159), # Single precision
        'float64': np.float64(3.14159265359), # Double precision
        'bool': np.bool_(True), # Boolean
        'complex64': np.complex64(1+2j), # Complex
        'unicode': np.unicode_('Hello') # Unicode string
    }

    # Pandas data types

```

```

df_example = pd.DataFrame({
    'integers': pd.Series([1, 2, 3, 4], dtype='int64'),
    'floats': pd.Series([1.1, 2.2, 3.3, 4.4], dtype='float64'),
    'categories': pd.Series(['A', 'B', 'A', 'C'], dtype='category'),
    'booleans': pd.Series([True, False, True, False], dtype='bool'),
    'dates': pd.Series(pd.date_range('2024-01-01', periods=4)),
    'strings': pd.Series(['apple', 'banana', 'cherry', 'date'], dtype='string'),
    'nullable_ints': pd.Series([1, 2, None, 4], dtype='Int64'), # Nullable integers
})

self.examples['data_science_types'] = {
    'numpy': numpy_types,
    'pandas': df_example
}

return self.examples['data_science_types']

def memory_usage_analysis(self):
    """Analyze memory usage of different data types"""

    # Create sample data
    n = 1000000

    # Different ways to store the same data
    data_variations = {
        'python_list': list(range(n)),
        'numpy_int32': np.arange(n, dtype=np.int32),
        'numpy_int64': np.arange(n, dtype=np.int64),
        'pandas_series_int32': pd.Series(range(n), dtype='int32'),
        'pandas_series_int64': pd.Series(range(n), dtype='int64'),
    }

    memory_usage = {}
    for name, data in data_variations.items():
        if isinstance(data, list):
            # Approximate memory usage for Python list
            memory_usage[name] = sys.getsizeof(data) + sum(sys.getsizeof(item) for item in data)
        elif isinstance(data, np.ndarray):
            memory_usage[name] = data.nbytes
        elif isinstance(data, pd.Series):
            memory_usage[name] = data.memory_usage(deep=True)

    # Convert to MB for readability
    memory_mb = {name: size / (1024**2) for name, size in memory_usage.items()}

    print("Memory Usage Comparison (MB):")
    for name, size in memory_mb.items():
        print(f" {name}: {size:.2f} MB")

    return memory_mb

def type_conversion_examples(self):
    """Demonstrate type conversion techniques"""

    # Sample data with mixed types
    mixed_data = pd.DataFrame({

```

```

        'numeric_strings': ['1', '2', '3', '4.5'],
        'dates_strings': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04'],
        'boolean_strings': ['True', 'False', '1', '0'],
        'categories': ['A', 'B', 'A', 'C'] * 25,  # Repeated values
        'mixed_numbers': [1, 2.5, '3', 4.0]
    })

    print("Original Data Types:")
    print(mixed_data.dtypes)
    print(f"Memory usage: {mixed_data.memory_usage(deep=True).sum()} bytes")

    # Convert types for better performance and functionality
    converted_data = mixed_data.copy()

    # Convert to appropriate types
    converted_data['numeric_strings'] = pd.to_numeric(converted_data['numeric_strings'])
    converted_data['dates_strings'] = pd.to_datetime(converted_data['dates_strings'])
    converted_data['boolean_strings'] = converted_data['boolean_strings'].map(
        {'True': True, 'False': False, '1': True, '0': False}
    )
    converted_data['categories'] = converted_data['categories'].astype('category')
    converted_data['mixed_numbers'] = pd.to_numeric(converted_data['mixed_numbers'])

    print("\nAfter Type Conversion:")
    print(converted_data.dtypes)
    print(f"Memory usage: {converted_data.memory_usage(deep=True).sum()} bytes")

    # Performance comparison
    import time

    # Test filtering performance
    start_time = time.time()
    result1 = mixed_data['categories'] == 'A'
    time1 = time.time() - start_time

    start_time = time.time()
    result2 = converted_data['categories'] == 'A'
    time2 = time.time() - start_time

    print(f"\nFiltering Performance:")
    print(f"  Original (object): {time1:.6f} seconds")
    print(f"  Categorical: {time2:.6f} seconds")
    print(f"  Speedup: {time1/time2:.2f}x")

    return converted_data

# Run demonstrations
data_types_demo = DataTypesDemo()

print("=== BASIC PYTHON DATA TYPES ===")
basic_types = data_types_demo.demonstrate_basic_types()

print("\n=== DATA SCIENCE SPECIFIC TYPES ===")
ds_types = data_types_demo.demonstrate_data_science_types()

print("\n=== MEMORY USAGE ANALYSIS ===")

```

```
memory_analysis = data_types_demo.memory_usage_analysis()

print("\n=== TYPE CONVERSION EXAMPLES ===")
conversion_example = data_types_demo.type_conversion_examples()
```

NumPy Interview Questions

Q3: What is NumPy and how does it differ from Python lists? Explain broadcasting.

Answer:

NumPy vs Python Lists:

Aspect	Python Lists	NumPy Arrays
Memory	Higher memory usage	Memory efficient
Speed	Slower for numerical operations	Optimized for numerical computations
Data Types	Can store mixed types	Homogeneous data types
Functionality	Basic operations	Rich mathematical functions
Dimensionality	Only 1D (nested for multi-D)	Multi-dimensional natively
Broadcasting	Not supported	Advanced broadcasting

Comprehensive NumPy Examples:

```
import numpy as np
import time
import matplotlib.pyplot as plt
from memory_profiler import profile

class NumPyMastery:
    def __init__(self):
        pass

    def performance_comparison(self):
        """Compare performance between lists and NumPy arrays"""

        # Create large datasets
        size = 1000000
        python_list1 = list(range(size))
        python_list2 = list(range(size, 2*size))

        numpy_array1 = np.arange(size)
        numpy_array2 = np.arange(size, 2*size)

        # Test addition operation
        print("Performance Comparison: Addition")
        print("-" * 40)
```

```

# Python lists
start_time = time.time()
result_list = [a + b for a, b in zip(python_list1, python_list2)]
list_time = time.time() - start_time
print(f"Python Lists: {list_time:.4f} seconds")

# NumPy arrays
start_time = time.time()
result_numpy = numpy_array1 + numpy_array2
numpy_time = time.time() - start_time
print(f"NumPy Arrays: {numpy_time:.4f} seconds")
print(f"NumPy is {list_time/numpy_time:.1f}x faster")

return list_time, numpy_time

def broadcasting_examples(self):
    """Comprehensive broadcasting examples"""

    print("\n=== NUMPY BROADCASTING EXAMPLES ===")

    # Example 1: Scalar with array
    arr1d = np.array([1, 2, 3, 4])
    scalar = 10
    result1 = arr1d + scalar
    print(f"1D Array + Scalar:")
    print(f" {arr1d} + {scalar} = {result1}")

    # Example 2: 1D array with 2D array
    arr2d = np.array([[1, 2, 3, 4],
                      [5, 6, 7, 8],
                      [9, 10, 11, 12]])

    result2 = arr2d + arr1d # Broadcasting along columns
    print(f"\n2D Array + 1D Array (row-wise):")
    print(f"Shape {arr2d.shape} + Shape {arr1d.shape}")
    print(f"Result shape: {result2.shape}")
    print(result2)

    # Example 3: Column vector with row vector
    col_vector = np.array([[1], [2], [3]]) # Shape (3, 1)
    row_vector = np.array([10, 20, 30, 40]) # Shape (4,)

    result3 = col_vector + row_vector
    print(f"\nColumn vector + Row vector:")
    print(f"Shape {col_vector.shape} + Shape {row_vector.shape}")
    print(f"Result shape: {result3.shape}")
    print(result3)

    # Example 4: Broadcasting rules demonstration
    self.demonstrate_broadcasting_rules()

    return result1, result2, result3

def demonstrate_broadcasting_rules(self):
    """Demonstrate NumPy broadcasting rules"""

```

```

print("\n=== BROADCASTING RULES ===")

# Rule 1: Arrays are aligned from the rightmost dimension
a = np.random.rand(3, 4, 5)
b = np.random.rand(4, 5)

try:
    result = a + b # This works
    print(f"✓ Shape {a.shape} + Shape {b.shape} = Shape {result.shape}")
except ValueError as e:
    print(f"✗ Broadcasting failed: {e}")

# Rule 2: Dimensions of size 1 can be stretched
c = np.random.rand(3, 1, 5)
d = np.random.rand(1, 4, 1)

try:
    result = c + d # This works
    print(f"✓ Shape {c.shape} + Shape {d.shape} = Shape {result.shape}")
except ValueError as e:
    print(f"✗ Broadcasting failed: {e}")

# Rule 3: Incompatible dimensions
e = np.random.rand(3, 4)
f = np.random.rand(2, 3)

try:
    result = e + f # This fails
    print(f"✓ Shape {e.shape} + Shape {f.shape} = Shape {result.shape}")
except ValueError as e:
    print(f"✗ Broadcasting failed: {e}")

def advanced_array_operations(self):
    """Advanced NumPy array operations"""

    # Array creation methods
    creation_methods = {
        'zeros': np.zeros((3, 4)),
        'ones': np.ones((2, 3)),
        'empty': np.empty((2, 2)),
        'arange': np.arange(0, 10, 2),
        'linspace': np.linspace(0, 1, 5),
        'logspace': np.logspace(0, 2, 5),
        'identity': np.identity(3),
        'random': np.random.rand(2, 3)
    }

    print("\n=== ARRAY CREATION METHODS ===")
    for name, array in creation_methods.items():
        print(f"{name}: {array}\n")

    # Array manipulation
    original = np.arange(24).reshape(2, 3, 4)

    manipulations = {
        'original_shape': original.shape,

```

```

        'flattened': original.flatten(),
        'reshaped': original.reshape(6, 4),
        'transposed': original.transpose(),
        'swapped_axes': np.swapaxes(original, 0, 2)
    }

    print("=== ARRAY MANIPULATION ===")
    for name, result in manipulations.items():
        if isinstance(result, tuple):
            print(f"{name}: {result}")
        else:
            print(f"{name} shape: {result.shape}")

    return creation_methods, manipulations

def universal_functions_demo(self):
    """Demonstrate NumPy universal functions (ufuncs)"""

    # Sample data
    x = np.array([-2, -1, 0, 1, 2])
    y = np.array([1, 2, 3, 4, 5])

    # Mathematical ufuncs
    math_ufuncs = {
        'abs': np.abs(x),
        'sqrt': np.sqrt(y),
        'exp': np.exp(x),
        'log': np.log(y),
        'sin': np.sin(x),
        'cos': np.cos(x),
        'power': np.power(x, 2)
    }

    # Comparison ufuncs
    comparison_ufuncs = {
        'greater': np.greater(y, 3),
        'equal': np.equal(x, 0),
        'logical_and': np.logical_and(x > -1, x < 1)
    }

    # Aggregation ufuncs
    aggregation_ufuncs = {
        'sum': np.sum(y),
        'mean': np.mean(y),
        'std': np.std(y),
        'min': np.min(y),
        'max': np.max(y),
        'argmin': np.argmin(y),
        'argmax': np.argmax(y)
    }

    print("\n=== UNIVERSAL FUNCTIONS ===")
    print(f"Original arrays: x={x}, y={y}")

    print("\nMathematical ufuncs:")
    for name, result in math_ufuncs.items():

```

```

        print(f" {name}: {result}")

    print("\nComparison ufuncs:")
    for name, result in comparison_ufuncs.items():
        print(f" {name}: {result}")

    print("\nAggregation ufuncs:")
    for name, result in aggregation_ufuncs.items():
        print(f" {name}: {result}")

    return math_ufuncs, comparison_ufuncs, aggregation_ufuncs

def linear_algebra_operations(self):
    """Linear algebra operations with NumPy"""

    # Create sample matrices
    A = np.random.rand(3, 3)
    B = np.random.rand(3, 3)
    v = np.random.rand(3)

    # Linear algebra operations
    linalg_ops = {
        'matrix_multiply': np.matmul(A, B),
        'dot_product': np.dot(v, v),
        'determinant': np.linalg.det(A),
        'inverse': np.linalg.inv(A),
        'eigenvalues': np.linalg.eigvals(A),
        'trace': np.trace(A),
        'rank': np.linalg.matrix_rank(A),
        'norm': np.linalg.norm(v)
    }

    print("\n=== LINEAR ALGEBRA OPERATIONS ===")
    for name, result in linalg_ops.items():
        if isinstance(result, np.ndarray) and result.ndim > 1:
            print(f"{name} shape: {result.shape}")
        else:
            print(f"{name}: {result}")

    # Solve linear system: Ax = b
    b = np.random.rand(3)
    x = np.linalg.solve(A, b)
    verification = np.allclose(np.dot(A, x), b)

    print(f"\nSolving linear system Ax = b:")
    print(f" Solution found: {verification}")
    print(f" Max error: {np.max(np.abs(np.dot(A, x) - b))}")

    return linalg_ops

# Comprehensive NumPy demonstration
numpy_master = NumPyMastery()

# Performance comparison
print("=== PERFORMANCE COMPARISON ===")
list_time, numpy_time = numpy_master.performance_comparison()

```

```
# Broadcasting examples
broadcasting_results = numpy_master.broadcasting_examples()

# Advanced operations
creation_ops, manipulation_ops = numpy_master.advanced_array_operations()

# Universal functions
math_funcs, comp_funcs, agg_funcs = numpy_master.universal_functions_demo()

# Linear algebra
linalg_results = numpy_master.linear_algebra_operations()
```

Q4: How do you handle missing values and data type conversions in NumPy?

Answer:

NumPy Missing Values and Data Type Handling:

```
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

class NumPyDataHandling:
    def __init__(self):
        pass

    def missing_values_demo(self):
        """Comprehensive missing value handling in NumPy"""

        print("=== NUMPY MISSING VALUES HANDLING ===")

        # Create array with missing values
        data_with_nan = np.array([1.0, 2.0, np.nan, 4.0, 5.0, np.nan])

        print(f"Original array: {data_with_nan}")
        print(f"Contains NaN: {np.isnan(data_with_nan).any()}")
        print(f"NaN positions: {np.where(np.isnan(data_with_nan))}")
        print(f"Number of NaNs: {np.isnan(data_with_nan).sum()}")

        # Different ways to handle missing values
        handling_methods = {}

        # Method 1: Remove NaN values
        handling_methods['remove_nan'] = data_with_nan[~np.isnan(data_with_nan)]

        # Method 2: Replace with mean (ignoring NaN)
        mean_value = np.nanmean(data_with_nan)
        handling_methods['replace_with_mean'] = np.where(
            np.isnan(data_with_nan), mean_value, data_with_nan
        )

        # Method 3: Replace with median
```

```

median_value = np.nanmedian(data_with_nan)
handling_methods['replace_with_median'] = np.where(
    np.isnan(data_with_nan), median_value, data_with_nan
)

# Method 4: Forward fill (carry forward last valid value)
forward_filled = data_with_nan.copy()
for i in range(1, len(forward_filled)):
    if np.isnan(forward_filled[i]):
        forward_filled[i] = forward_filled[i-1]
handling_methods['forward_fill'] = forward_filled

# Method 5: Linear interpolation
def linear_interpolate(arr):
    valid_indices = ~np.isnan(arr)
    if valid_indices.sum() < 2:
        return arr

    result = arr.copy()
    x_valid = np.where(valid_indices)[0]
    y_valid = arr[valid_indices]

    # Interpolate missing values
    x_missing = np.where(~valid_indices)[0]
    result[x_missing] = np.interp(x_missing, x_valid, y_valid)

    return result

handling_methods['linear_interpolation'] = linear_interpolate(data_with_nan)

print("\nMissing value handling methods:")
for method, result in handling_methods.items():
    print(f" {method}: {result}")

return handling_methods

def nan_aware_operations(self):
    """Demonstrate NaN-aware NumPy functions"""

    # 2D array with missing values
    data_2d = np.array([
        [1.0, 2.0, np.nan, 4.0],
        [5.0, np.nan, 7.0, 8.0],
        [np.nan, 10.0, 11.0, 12.0]
    ])

    print("\n=== NAN-AWARE OPERATIONS ===")
    print(f"Original 2D array:\n{data_2d}")

    # NaN-aware statistical functions
    nan_stats = {
        'nanmean': np.nanmean(data_2d),
        'nanmedian': np.nanmedian(data_2d),
        'nanstd': np.nanstd(data_2d),
        'nanvar': np.nanvar(data_2d),
        'nanmin': np.nanmin(data_2d),
    }

```

```

        'nanmax': np.nanmax(data_2d),
        'nansum': np.nansum(data_2d),
        'nanprod': np.nanprod(data_2d)
    }

    # Axis-specific operations
    axis_stats = {
        'nanmean_axis0': np.nanmean(data_2d, axis=0), # Column-wise
        'nanmean_axis1': np.nanmean(data_2d, axis=1), # Row-wise
        'nansum_axis0': np.nansum(data_2d, axis=0),
        'nansum_axis1': np.nansum(data_2d, axis=1)
    }

    print("\nNaN-aware statistics:")
    for func_name, result in nan_stats.items():
        print(f" {func_name}: {result:.4f}")

    print("\nAxis-specific NaN-aware operations:")
    for func_name, result in axis_stats.items():
        print(f" {func_name}: {result}")

    return nan_stats, axis_stats

def data_type_conversions(self):
    """Comprehensive data type conversion examples"""

    print("\n=== DATA TYPE CONVERSIONS ===")

    # Original array with mixed-type data as strings
    string_data = np.array(['1', '2.5', '3', '4.7', '5'])
    print(f"Original string array: {string_data}")
    print(f>Data type: {string_data.dtype}")

    # Conversion methods
    conversions = {}

    # Method 1: astype() - explicit conversion
    conversions['to_float'] = string_data.astype(np.float64)
    conversions['to_int'] = string_data.astype(np.float64).astype(np.int32) # Two-st

    # Method 2: Using array constructor with dtype
    conversions['constructor_float'] = np.array(string_data, dtype=np.float32)

    # Method 3: Safe conversion with error handling
    def safe_convert_to_numeric(arr, target_dtype=np.float64, fill_value=0):
        result = np.full(arr.shape, fill_value, dtype=target_dtype)
        for i, val in enumerate(arr):
            try:
                result[i] = target_dtype(val)
            except (ValueError, TypeError):
                pass # Keep fill_value
        return result

    # Test with problematic data
    problematic_data = np.array(['1', '2.5', 'invalid', '4.7', '5'])
    conversions['safe_conversion'] = safe_convert_to_numeric(

```

```

        problematic_data, np.float64, np.nan
    )

    print("\nData type conversions:")
    for method, result in conversions.items():
        print(f" {method}: {result} (dtype: {result.dtype})")

    # Demonstrate different numeric dtypes and their ranges
    self.demonstrate_numeric_dtypes()

    return conversions

def demonstrate_numeric_dtypes(self):
    """Show different numeric dtypes and their properties"""

    print("\n=== NUMERIC DATA TYPES ===")

    # Integer types
    int_types = [np.int8, np.int16, np.int32, np.int64]
    uint_types = [np.uint8, np.uint16, np.uint32, np.uint64]
    float_types = [np.float16, np.float32, np.float64]

    print("Integer types:")
    for dtype in int_types:
        info = np.iinfo(dtype)
        print(f" {dtype.__name__}: {info.bits} bits, range [{info.min}, {info.max}]")

    print("\nUnsigned integer types:")
    for dtype in uint_types:
        info = np.iinfo(dtype)
        print(f" {dtype.__name__}: {info.bits} bits, range [{info.min}, {info.max}]")

    print("\nFloat types:")
    for dtype in float_types:
        info = np.finfo(dtype)
        print(f" {dtype.__name__}: {info.bits} bits, precision: {info.precision}")

    # Memory efficiency demonstration
    data = np.arange(1000000)

    memory_usage = {}
    for dtype in [np.int8, np.int32, np.int64, np.float32, np.float64]:
        try:
            converted = data.astype(dtype)
            memory_usage[dtype.__name__] = converted.nbytes / (1024**2) # MB
        except OverflowError:
            memory_usage[dtype.__name__] = "Overflow"

    print(f"\nMemory usage for 1M integers:")
    for dtype_name, memory in memory_usage.items():
        if isinstance(memory, str):
            print(f" {dtype_name}: {memory}")
        else:
            print(f" {dtype_name}: {memory:.2f} MB")

def advanced_indexing_with_missing_values(self):

```

```

"""Advanced indexing techniques with missing values"""

print("\n=== ADVANCED INDEXING WITH MISSING VALUES ===")

# Create complex array with missing values
np.random.seed(42)
data = np.random.randn(5, 4)
# Introduce some missing values
data[1, 2] = np.nan
data[3, 0] = np.nan
data[4, 3] = np.nan

print(f"Original array:\n{data}")

# Boolean indexing to find missing values
missing_mask = np.isnan(data)
print(f"\nMissing value mask:\n{missing_mask}")

# Find rows with any missing values
rows_with_nan = np.any(missing_mask, axis=1)
print(f"Rows with missing values: {np.where(rows_with_nan)[0]}")

# Find columns with any missing values
cols_with_nan = np.any(missing_mask, axis=0)
print(f"Columns with missing values: {np.where(cols_with_nan)[0]}")

# Extract complete cases (rows without any missing values)
complete_cases = data[~rows_with_nan]
print(f"\nComplete cases:\n{complete_cases}")

# Replace missing values with column means
col_means = np.nanmean(data, axis=0)
data_filled = data.copy()
for col in range(data.shape[1]):
    col_mask = missing_mask[:, col]
    data_filled[col_mask, col] = col_means[col]

print(f"\nData with missing values filled with column means:\n{data_filled}")

return data, missing_mask, complete_cases, data_filled

# Comprehensive demonstration
handler = NumPyDataHandling()

# Missing values handling
missing_methods = handler.missing_values_demo()

# NaN-aware operations
nan_stats, axis_stats = handler.nan_aware_operations()

# Data type conversions
conversion_examples = handler.data_type_conversions()

# Advanced indexing with missing values
data, mask, complete, filled = handler.advanced_indexing_with_missing_values()

```

Pandas Interview Questions

Q5: Explain Pandas data structures (Series and DataFrame) and their key operations.

Answer:

Pandas Data Structures Comprehensive Guide:

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

class PandasMastery:
    def __init__(self):
        pass

    def series_comprehensive_demo(self):
        """Comprehensive Series operations"""

        print("=== PANDAS SERIES OPERATIONS ===")

        # Creating Series in different ways
        creation_methods = {}

        # Method 1: From list
        creation_methods['from_list'] = pd.Series([1, 2, 3, 4, 5])

        # Method 2: From dictionary
        creation_methods['from_dict'] = pd.Series({
            'apple': 10, 'banana': 20, 'cherry': 15
        })

        # Method 3: From NumPy array with custom index
        creation_methods['from_numpy'] = pd.Series(
            np.random.randn(5),
            index=['A', 'B', 'C', 'D', 'E']
        )

        # Method 4: From scalar value
        creation_methods['from_scalar'] = pd.Series(
            5.0, index=['x', 'y', 'z']
        )

        # Method 5: Date range series
        creation_methods['date_series'] = pd.Series(
            pd.date_range('2024-01-01', periods=5, freq='D')
        )

        print("Series Creation Methods:")
        for method, series in creation_methods.items():
            print(f"\n{method}:")
```

```

        print(series)
        print(f"  Index: {series.index.tolist()}")
        print(f"  Values: {series.values}")
        print(f"  Data type: {series.dtype}")

# Series operations
numeric_series = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

operations = {
    'head_3': numeric_series.head(3),
    'tail_2': numeric_series.tail(2),
    'describe': numeric_series.describe(),
    'value_counts': pd.Series(['A', 'B', 'A', 'C', 'B']).value_counts(),
    'unique': pd.Series(['A', 'B', 'A', 'C', 'B']).unique(),
    'nunique': pd.Series(['A', 'B', 'A', 'C', 'B']).nunique()
}

print("\n=== SERIES OPERATIONS ===")
for op_name, result in operations.items():
    print(f"\n{op_name}:")
    print(result)

return creation_methods, operations

def dataframe_comprehensive_demo(self):
    """Comprehensive DataFrame operations"""

    print("\n=== PANDAS DATAFRAME OPERATIONS ===")

    # Creating DataFrames
    np.random.seed(42)

    # Method 1: From dictionary
    df_from_dict = pd.DataFrame({
        'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
        'age': [25, 30, 35, 28, 32],
        'salary': [50000, 60000, 70000, 55000, 65000],
        'department': ['IT', 'HR', 'IT', 'Finance', 'IT']
    })

    # Method 2: From list of dictionaries
    df_from_list = pd.DataFrame([
        {'product': 'A', 'price': 100, 'quantity': 5},
        {'product': 'B', 'price': 150, 'quantity': 3},
        {'product': 'C', 'price': 200, 'quantity': 2}
    ])

    # Method 3: From NumPy array
    df_from_numpy = pd.DataFrame(
        np.random.randn(4, 3),
        columns=['col1', 'col2', 'col3'],
        index=['row1', 'row2', 'row3', 'row4']
    )

    print("DataFrame Creation:")
    print("\nFrom dictionary:")

```

```

print(df_from_dict)
print(f"Shape: {df_from_dict.shape}")
print(f"Columns: {df_from_dict.columns.tolist()}")
print(f"Index: {df_from_dict.index.tolist()}")

# DataFrame information
print(f"\nDataFrame Info:")
print(f>Data types:\n{df_from_dict.dtypes}")
print(f"Memory usage:\n{df_from_dict.memory_usage(deep=True)}")
print(f"Missing values:\n{df_from_dict.isnull().sum()}")

return df_from_dict, df_from_list, df_from_numpy

def indexing_and_selection(self, df):
    """Comprehensive indexing and selection operations"""

    print("\n=== INDEXING AND SELECTION ===")

    # Column selection
    print("Column Selection:")
    print(f"Single column:\n{df['name']}")
    print(f"\nMultiple columns:\n{df[['name', 'age']]}")

    # Row selection
    print("\nRow Selection:")
    print(f"iloc (integer-based):\n{df.iloc[0:2]}") # First 2 rows
    print(f"loc (label-based):\n{df.loc[0:2, ['name', 'age']]}") # Inclusive

    # Boolean indexing
    print("\nBoolean Indexing:")
    high_salary = df[df['salary'] > 60000]
    print(f"High salary employees:\n{high_salary}")

    # Complex conditions
    it_high_salary = df[(df['department'] == 'IT') & (df['salary'] > 55000)]
    print(f"\nIT employees with high salary:\n{it_high_salary}")

    # Query method (alternative to boolean indexing)
    query_result = df.query('age > 30 and department == "IT"')
    print(f"\nUsing query method:\n{query_result}")

    # Advanced indexing
    df_indexed = df.set_index('name')
    print(f"\nDataFrame with name as index:\n{df_indexed}")
    print(f"Access by index:\n{df_indexed.loc['Alice']}")

    return high_salary, it_high_salary, df_indexed

def data_manipulation_operations(self, df):
    """Data manipulation and transformation operations"""

    print("\n=== DATA MANIPULATION ===")

    # Adding new columns
    df_copy = df.copy()

```

```

# Method 1: Simple assignment
df_copy['bonus'] = df_copy['salary'] * 0.1

# Method 2: Conditional assignment
df_copy['seniority'] = df_copy['age'].apply(
    lambda x: 'Senior' if x >= 30 else 'Junior'
)

# Method 3: Using np.where for conditional assignment
df_copy['performance'] = np.where(
    df_copy['salary'] > df_copy['salary'].median(),
    'High', 'Low'
)

# Method 4: Using map with dictionary
dept_mapping = {'IT': 'Technology', 'HR': 'Human Resources', 'Finance': 'Financial'}
df_copy['full_department'] = df_copy['department'].map(dept_mapping)

print("DataFrame with new columns:")
print(df_copy)

# Sorting operations
print("\nSorting Operations:")
sorted_by_age = df_copy.sort_values('age', ascending=False)
print(f"Sorted by age (descending):\n{sorted_by_age[['name', 'age']]}")

sorted_multiple = df_copy.sort_values(['department', 'salary'],
                                       ascending=[True, False])
print(f"\nSorted by department and salary:\n{sorted_multiple[['name', 'department', 'salary']]}")

# Groupby operations
print("\nGroupBy Operations:")
dept_stats = df_copy.groupby('department').agg({
    'salary': ['mean', 'sum', 'count'],
    'age': ['mean', 'min', 'max']
})
print(dept_stats)

# Custom aggregation
custom_agg = df_copy.groupby('department').apply(
    lambda x: pd.Series({
        'avg_salary': x['salary'].mean(),
        'salary_range': x['salary'].max() - x['salary'].min(),
        'employees': len(x)
    })
)
print(f"\nCustom aggregation:\n{custom_agg}")

return df_copy, dept_stats, custom_agg

def missing_data_handling(self):
    """Comprehensive missing data handling"""

    print("\n=== MISSING DATA HANDLING ===")

    # Create DataFrame with missing values

```

```

data_with_missing = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5],
    'B': [1, np.nan, 3, np.nan, 5],
    'C': ['a', 'b', np.nan, 'd', 'e'],
    'D': pd.date_range('2024-01-01', periods=5),
    'E': [np.nan, np.nan, np.nan, np.nan, np.nan]
})

print("DataFrame with missing values:")
print(data_with_missing)
print(f"\nMissing value info:")
print(data_with_missing.isnull().sum())

# Different handling strategies
strategies = {}

# Strategy 1: Drop rows with any missing values
strategies['drop_rows'] = data_with_missing.dropna()

# Strategy 2: Drop columns with any missing values
strategies['drop_cols'] = data_with_missing.dropna(axis=1)

# Strategy 3: Fill with specific values
strategies['fill_specific'] = data_with_missing.fillna({
    'A': data_with_missing['A'].mean(),
    'B': data_with_missing['B'].median(),
    'C': 'Unknown'
})

# Strategy 4: Forward fill
strategies['forward_fill'] = data_with_missing.fillna(method='ffill')

# Strategy 5: Backward fill
strategies['backward_fill'] = data_with_missing.fillna(method='bfill')

# Strategy 6: Interpolation
numeric_cols = data_with_missing.select_dtypes(include=[np.number]).columns
strategies['interpolate'] = data_with_missing.copy()
strategies['interpolate'][numeric_cols] = data_with_missing[numeric_cols].interpolate()

print("\nMissing data handling strategies:")
for strategy, result in strategies.items():
    print(f"\n{strategy} shape: {result.shape}")
    print(f"Missing values: {result.isnull().sum().sum()}")

return data_with_missing, strategies

def advanced_operations(self, df):
    """Advanced DataFrame operations"""

    print("\n=== ADVANCED OPERATIONS ===")

    # Pivot tables
    print("Pivot Table:")
    pivot_data = pd.DataFrame({
        'date': pd.date_range('2024-01-01', periods=12, freq='M'),

```

```

        'region': ['North', 'South'] * 6,
        'product': ['A', 'B'] * 6,
        'sales': np.random.randint(100, 1000, 12)
    })

    pivot_table = pivot_data.pivot_table(
        values='sales',
        index='region',
        columns='product',
        aggfunc=['mean', 'sum']
    )
    print(pivot_table)

    # Melt operation (unpivot)
    print("\nMelt Operation:")
    melted = pd.melt(df,
                     id_vars=['name'],
                     value_vars=['age', 'salary'],
                     var_name='metric',
                     value_name='value')
    print(melted)

    # Crosstab
    print("\nCrosstab:")
    crosstab_result = pd.crosstab(df['department'], df['age'] > 30, margins=True)
    print(crosstab_result)

    # Window operations
    print("\nWindow Operations:")
    df_copy = df.copy().sort_values('salary')
    df_copy['salary_rank'] = df_copy['salary'].rank()
    df_copy['salary_pct_rank'] = df_copy['salary'].rank(pct=True)
    df_copy['salary_rolling_mean'] = df_copy['salary'].rolling(window=2).mean()

    print(df_copy[['name', 'salary', 'salary_rank', 'salary_pct_rank', 'salary_rolling_mean']])

    return pivot_table, melted, crosstab_result, df_copy

# Comprehensive Pandas demonstration
pandas_master = PandasMastery()

# Series operations
series_creation, series_operations = pandas_master.series_comprehensive_demo()

# DataFrame operations
df_dict, df_list, df_numpy = pandas_master.dataframe_comprehensive_demo()

# Use the main DataFrame for further operations
main_df = df_dict

# Indexing and selection
high_sal, it_high_sal, indexed_df = pandas_master.indexing_and_selection(main_df)

# Data manipulation
manipulated_df, dept_stats, custom_agg = pandas_master.data_manipulation_operations(main_df,

```

```
# Missing data handling
missing_df, strategies = pandas_master.missing_data_handling()

# Advanced operations
pivot, melted, crosstab, windowed = pandas_master.advanced_operations(main_df)
```

Q6: How do you perform data merging and joining operations in Pandas?

Answer:

Comprehensive Data Merging and Joining:

```
import pandas as pd
import numpy as np

class PandasMergeJoin:
    def __init__(self):
        self.create_sample_data()

    def create_sample_data(self):
        """Create sample datasets for merging examples"""

        # Employees dataset
        self.employees = pd.DataFrame({
            'emp_id': [1, 2, 3, 4, 5],
            'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
            'department_id': [10, 20, 10, 30, 20],
            'salary': [50000, 60000, 55000, 70000, 65000],
            'join_date': pd.to_datetime(['2020-01-15', '2019-05-20', '2021-03-10', '2018-
        })

        # Departments dataset
        self.departments = pd.DataFrame({
            'dept_id': [10, 20, 30, 40],
            'dept_name': ['Engineering', 'Marketing', 'Finance', 'HR'],
            'manager': ['John', 'Jane', 'Mike', 'Sarah'],
            'budget': [1000000, 500000, 750000, 300000]
        })

        # Projects dataset
        self.projects = pd.DataFrame({
            'project_id': [101, 102, 103, 104, 105],
            'emp_id': [1, 2, 1, 3, 6], # Note: emp_id 6 doesn't exist in employees
            'project_name': ['WebApp', 'Mobile', 'Database', 'API', 'Analytics'],
            'hours': [40, 35, 30, 25, 45]
        })

        # Employee skills (many-to-many relationship)
        self.skills = pd.DataFrame({
            'emp_id': [1, 1, 2, 2, 3, 4, 4, 5],
            'skill': ['Python', 'SQL', 'Marketing', 'Analytics', 'Python', 'Finance', 'Ex

        })

        print("=== SAMPLE DATASETS ===")
```

```

print("Employees:")
print(self.employees)
print("\nDepartments:")
print(self.departments)
print("\nProjects:")
print(self.projects)
print("\nSkills:")
print(self.skills)

def merge_operations_demo(self):
    """Demonstrate different merge operations"""

    print("\n=== MERGE OPERATIONS ===")

    # Inner merge (default)
    inner_merge = pd.merge(
        self.employees,
        self.departments,
        left_on='department_id',
        right_on='dept_id',
        how='inner'
    )
    print("Inner Merge (employees with departments):")
    print(inner_merge[['name', 'dept_name', 'salary', 'manager']])

    # Left merge
    left_merge = pd.merge(
        self.employees,
        self.projects,
        on='emp_id',
        how='left'
    )
    print("\nLeft Merge (all employees, with their projects if any):")
    print(left_merge[['name', 'project_name', 'hours']])

    # Right merge
    right_merge = pd.merge(
        self.employees,
        self.projects,
        on='emp_id',
        how='right'
    )
    print("\nRight Merge (all projects, with employee info if available):")
    print(right_merge[['name', 'project_name', 'hours']])

    # Outer merge
    outer_merge = pd.merge(
        self.employees,
        self.projects,
        on='emp_id',
        how='outer',
        indicator=True # Shows source of each row
    )
    print("\nOuter Merge (all employees and projects):")
    print(outer_merge[['name', 'project_name', 'hours', '_merge']])

```

```

        return inner_merge, left_merge, right_merge, outer_merge

def join_operations_demo(self):
    """Demonstrate DataFrame join operations"""

    print("\n=== JOIN OPERATIONS ===")

    # Set index for joining
    emp_indexed = self.employees.set_index('emp_id')
    proj_indexed = self.projects.set_index('emp_id')

    # Inner join
    inner_join = emp_indexed.join(proj_indexed, how='inner', rsuffix='_proj')
    print("Inner Join:")
    print(inner_join[['name', 'project_name', 'hours']])

    # Left join
    left_join = emp_indexed.join(proj_indexed, how='left', rsuffix='_proj')
    print("\nLeft Join:")
    print(left_join[['name', 'project_name', 'hours']])

    # Multiple DataFrame joins
    dept_indexed = self.departments.set_index('dept_id')

    # Chain joins
    multi_join = (emp_indexed
                  .join(dept_indexed, on='department_id', how='left')
                  .join(proj_indexed, how='left', rsuffix='_proj'))

    print("\nMultiple DataFrame Join:")
    print(multi_join[['name', 'dept_name', 'manager', 'project_name', 'hours']])

    return inner_join, left_join, multi_join

def concat_operations_demo(self):
    """Demonstrate concatenation operations"""

    print("\n=== CONCATENATION OPERATIONS ===")

    # Create additional datasets for concat demo
    new_employees = pd.DataFrame({
        'emp_id': [6, 7, 8],
        'name': ['Frank', 'Grace', 'Henry'],
        'department_id': [10, 30, 20],
        'salary': [58000, 72000, 61000],
        'join_date': pd.to_datetime(['2022-01-15', '2021-12-01', '2022-03-20'])
    })

    employee_bonuses = pd.DataFrame({
        'emp_id': [1, 2, 3, 4, 5],
        'bonus_2023': [5000, 6000, 5500, 7000, 6500],
        'bonus_2022': [4500, 5500, 5000, 6500, 6000]
    })

    # Vertical concatenation (adding rows)
    all_employees = pd.concat([self.employees, new_employees], ignore_index=True)

```

```

print("Vertical Concatenation (adding new employees):")
print(all_employees)

# Horizontal concatenation (adding columns)
emp_with_bonuses = pd.concat([self.employees, employee_bonuses.iloc[:, 1:]], axis=1)
print("\nHorizontal Concatenation (adding bonus columns):")
print(emp_with_bonuses[['name', 'salary', 'bonus_2023', 'bonus_2022']])

# Concatenation with keys
dept_employees = {}
for dept_id in self.employees['department_id'].unique():
    dept_employees[dept_id] = self.employees[self.employees['department_id'] == dept_id]

concat_with_keys = pd.concat(dept_employees, names=['department_id', 'row_id'])
print("\nConcatenation with keys (hierarchical index):")
print(concat_with_keys[['name', 'salary']])

return all_employees, emp_with_bonuses, concat_with_keys

def advanced_merging_scenarios(self):
    """Advanced merging scenarios and techniques"""

    print("\n=== ADVANCED MERGING SCENARIOS ===")

    # Scenario 1: Multiple column merging
    # Create extended employee info
    emp_extended = pd.DataFrame({
        'first_name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
        'last_name': ['Smith', 'Johnson', 'Brown', 'Wilson', 'Davis'],
        'department_id': [10, 20, 10, 30, 20],
        'performance_rating': [4.5, 4.2, 4.8, 4.7, 4.3]
    })

    # Create department ratings
    dept_ratings = pd.DataFrame({
        'dept_id': [10, 20, 30],
        'dept_name': ['Engineering', 'Marketing', 'Finance'],
        'avg_rating': [4.6, 4.1, 4.7]
    })

    multi_col_merge = pd.merge(
        emp_extended,
        dept_ratings,
        left_on='department_id',
        right_on='dept_id',
        how='inner'
    )
    print("Multiple column merge:")
    print(multi_col_merge)

    # Scenario 2: Fuzzy matching (approximate string matching)
    emp_names_typos = pd.DataFrame({
        'name_approx': ['Alyce', 'Robert', 'Charlie', 'Dianna'],
        'external_id': ['EXT001', 'EXT002', 'EXT003', 'EXT004']
    })

```

```

# For demonstration, we'll use a simple approach
# In practice, you'd use libraries like fuzzywuzzy
def simple_fuzzy_match(name1, name2, threshold=0.8):
    """Simple fuzzy matching based on character overlap"""
    set1, set2 = set(name1.lower()), set(name2.lower())
    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))
    return intersection / union >= threshold

# Manual fuzzy merge (simplified)
fuzzy_matches = []
for _, row1 in self.employees.iterrows():
    for _, row2 in emp_names_typos.iterrows():
        if simple_fuzzy_match(row1['name'], row2['name_approx']):
            fuzzy_matches.append({
                'emp_id': row1['emp_id'],
                'name': row1['name'],
                'name_approx': row2['name_approx'],
                'external_id': row2['external_id']
            })

fuzzy_result = pd.DataFrame(fuzzy_matches)
print("\nFuzzy matching result:")
print(fuzzy_result)

# Scenario 3: Merge with validation
def merge_with_validation(left_df, right_df, on, how='inner'):
    """Merge with data validation"""

    # Check for duplicate keys
    left_dupes = left_df[on].duplicated().sum()
    right_dupes = right_df[on].duplicated().sum()

    if left_dupes > 0:
        print(f"Warning: {left_dupes} duplicate keys in left DataFrame")
    if right_dupes > 0:
        print(f"Warning: {right_dupes} duplicate keys in right DataFrame")

    # Perform merge
    result = pd.merge(left_df, right_df, on=on, how=how, indicator=True)

    # Report merge statistics
    merge_stats = result['_merge'].value_counts()
    print(f"Merge statistics:")
    for stat, count in merge_stats.items():
        print(f"  {stat}: {count}")

    return result

validated_merge = merge_with_validation(
    self.employees,
    self.departments,
    left_on='department_id',
    right_on='dept_id'
)

```

```

        return multi_col_merge, fuzzy_result, validated_merge

def performance_optimization_tips(self):
    """Performance optimization techniques for large datasets"""

    print("\n=== PERFORMANCE OPTIMIZATION TIPS ===")

    # Tip 1: Sort values before merging for better performance
    large_df1 = pd.DataFrame({
        'key': np.random.randint(0, 1000, 10000),
        'value1': np.random.randn(10000)
    }).sort_values('key')

    large_df2 = pd.DataFrame({
        'key': np.random.randint(0, 1000, 5000),
        'value2': np.random.randn(5000)
    }).sort_values('key')

    print("For large datasets:")
    print("1. Sort DataFrames by merge key beforehand")
    print("2. Use appropriate merge types (inner vs outer)")
    print("3. Consider using categorical data types for repeated values")
    print("4. Use chunks for very large datasets that don't fit in memory")

    # Tip 2: Memory efficient merging
    print("\nMemory efficient techniques:")
    print("- Use pd.merge() with 'how' parameter wisely")
    print("- Consider using suffixes to avoid column name conflicts")
    print("- Drop unnecessary columns before merging")
    print("- Use appropriate data types (int32 vs int64, category vs object)")

    # Example of memory optimization
    optimized_df = self.employees.copy()

    # Convert to categorical for repeated values
    optimized_df['department_id'] = optimized_df['department_id'].astype('category')

    print(f"\nMemory usage comparison:")
    print(f"Original: {self.employees.memory_usage(deep=True).sum()} bytes")
    print(f"Optimized: {optimized_df.memory_usage(deep=True).sum()} bytes")

    return large_df1, large_df2, optimized_df

# Comprehensive demonstration
merger = PandasMergeJoin()

# Merge operations
inner, left, right, outer = merger.merge_operations_demo()

# Join operations
inner_join, left_join, multi_join = merger.join_operations_demo()

# Concatenation operations
all_emp, emp_bonuses, keyed_concat = merger.concat_operations_demo()

# Advanced scenarios

```

```

multi_merge, fuzzy, validated = merger.advanced_merging_scenarios()

# Performance tips
large1, large2, optimized = merger.performance_optimization_tips()

```

Data Manipulation & Cleaning

Q7: How do you handle data cleaning and preprocessing tasks in Python?

Answer:

Comprehensive Data Cleaning Pipeline:

```

import pandas as pd
import numpy as np
import re
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

class DataCleaningPipeline:
    def __init__(self):
        self.cleaning_report = {}

    def create_dirty_dataset(self):
        """Create a dataset with common data quality issues"""

        np.random.seed(42)
        n_records = 1000

        # Generate base data with intentional issues
        data = {
            # Names with inconsistent formatting
            'customer_name': [
                np.random.choice([
                    'john smith', 'JANE DOE', 'Bob Johnson ', ' Alice Brown',
                    'charlie wilson', 'DIANA DAVIS', None, '', ' ', 'N/A'
                ]) for _ in range(n_records)
            ],

            # Emails with various issues
            'email': [
                np.random.choice([
                    'john@email.com', 'JANE@COMPANY.COM', 'bob.johnson@test',
                    'alice@', '@company.com', 'invalid-email', None, '',
                    'charlie@domain.co.uk', 'diana.davis@company123.com'
                ]) for _ in range(n_records)
            ],

            # Phone numbers in various formats
            'phone': [
                np.random.choice([
                    '(555) 123-4567', '555-123-4567', '5551234567',

```

```

        '555.123.4567', '+1-555-123-4567', '555 123 4567',
        '123-456', '555-123-45678', None, '', 'N/A'
    ]) for _ in range(n_records)
],

# Ages with outliers and missing values
'age': np.random.choice(
    [18, 25, 30, 35, 40, 45, 50, 55, 60, 65, 120, -5, 200, None, 0],
    n_records
),

# Salaries with outliers
'salary': np.random.choice(
    [30000, 45000, 60000, 75000, 90000, 120000, 150000,
     -1000, 5000000, None, 0],
    n_records
),

# Dates in various formats
'join_date': [
    np.random.choice([
        '2023-01-15', '01/15/2023', '15-Jan-2023',
        '2023/01/15', 'January 15, 2023', '15/01/2023',
        '2030-01-15', '1999-01-15', 'invalid-date', None, ''
    ]) for _ in range(n_records)
],

# Categories with typos and inconsistencies
'department': [
    np.random.choice([
        'Sales', 'sales', 'SALES', 'Marketing', 'marketing',
        'Engineering', 'engineering', 'HR', 'hr', 'Human Resources',
        'Finance', 'finance', 'IT', 'it', None, '', 'Unknown'
    ]) for _ in range(n_records)
]
}

df = pd.DataFrame(data)

# Add some duplicate rows
duplicates = df.sample(n=50, random_state=42)
df = pd.concat([df, duplicates], ignore_index=True)

print("=== ORIGINAL DIRTY DATASET ===")
print(f"Shape: {df.shape}")
print(f>Data types:\n{df.dtypes}")
print(f>Missing values:\n{df.isnull().sum()}")
print(f>Sample data:\n{df.head()}")

return df

def assess_data_quality(self, df):
    """Comprehensive data quality assessment"""

    quality_issues = {}

```

```

for column in df.columns:
    column_issues = []

    # Missing values
    missing_count = df[column].isnull().sum()
    if missing_count > 0:
        column_issues.append(f"Missing: {missing_count} ({missing_count/len(df)*100}% of rows)")

    # Empty strings
    if df[column].dtype == 'object':
        empty_strings = (df[column] == '').sum()
        whitespace_only = df[column].str.strip().eq('').sum() if df[column].dtype == 'object' else 0

        if empty_strings > 0:
            column_issues.append(f"Empty strings: {empty_strings}")
        if whitespace_only > empty_strings:
            column_issues.append(f"Whitespace only: {whitespace_only - empty_strings}")

    # Check for placeholder values
    placeholders = df[column].str.lower().isin(['n/a', 'na', 'null', 'none']).sum()
    if placeholders > 0:
        column_issues.append(f"Placeholder values: {placeholders}")

    # Duplicates
    if column == df.columns[0]: # Check row duplicates once
        duplicate_rows = df.duplicated().sum()
        if duplicate_rows > 0:
            column_issues.append(f"Duplicate rows: {duplicate_rows}")

    # Column-specific checks
    if 'email' in column.lower():
        invalid_emails = self._count_invalid_emails(df[column])
        if invalid_emails > 0:
            column_issues.append(f"Invalid emails: {invalid_emails}")

    elif 'phone' in column.lower():
        invalid_phones = self._count_invalid_phones(df[column])
        if invalid_phones > 0:
            column_issues.append(f"Invalid phones: {invalid_phones}")

    elif 'age' in column.lower():
        invalid_ages = ((df[column] < 0) | (df[column] > 100)).sum()
        if invalid_ages > 0:
            column_issues.append(f"Invalid ages: {invalid_ages}")

    if column_issues:
        quality_issues[column] = column_issues

print("\n=== DATA QUALITY ASSESSMENT ===")
for column, issues in quality_issues.items():
    print(f"\n{column}:")
    for issue in issues:
        print(f"    - {issue}")

self.cleaning_report['quality_issues'] = quality_issues
return quality_issues

```

```

def clean_text_data(self, df):
    """Clean text columns (names, emails, etc.)"""

    df_cleaned = df.copy()

    # Clean customer names
    if 'customer_name' in df_cleaned.columns:
        print("\n=== CLEANING CUSTOMER NAMES ===")

        # Replace placeholder values with NaN
        df_cleaned['customer_name'] = df_cleaned['customer_name'].replace({
            'N/A': np.nan, 'n/a': np.nan, 'NA': np.nan, ' ': np.nan, ' ': np.nan
        })

        # Strip whitespace and convert to title case
        df_cleaned['customer_name'] = (df_cleaned['customer_name']
                                       .str.strip()
                                       .str.title())

        print(f"Cleaned names sample:")
        print(df_cleaned['customer_name'].dropna().head(10).tolist())

    # Clean email addresses
    if 'email' in df_cleaned.columns:
        print("\n=== CLEANING EMAIL ADDRESSES ===")

        # Convert to lowercase and strip whitespace
        df_cleaned['email'] = (df_cleaned['email']
                              .str.lower()
                              .str.strip())

        # Mark invalid emails as NaN
        email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        valid_emails = df_cleaned['email'].str.match(email_pattern, na=False)
        df_cleaned.loc[~valid_emails, 'email'] = np.nan

        print(f"Valid emails sample:")
        print(df_cleaned['email'].dropna().head(10).tolist())

    # Clean phone numbers
    if 'phone' in df_cleaned.columns:
        print("\n=== CLEANING PHONE NUMBERS ===")

        df_cleaned['phone_cleaned'] = self._standardize_phone_numbers(df_cleaned['phone'])

        print(f"Standardized phones sample:")
        print(df_cleaned['phone_cleaned'].dropna().head(10).tolist())

    # Clean department names
    if 'department' in df_cleaned.columns:
        print("\n=== CLEANING DEPARTMENT NAMES ===")

        # Standardize department names
        dept_mapping = {
            'sales': 'Sales',

```

```

        'SALES': 'Sales',
        'marketing': 'Marketing',
        'MARKETING': 'Marketing',
        'engineering': 'Engineering',
        'ENGINEERING': 'Engineering',
        'hr': 'HR',
        'HR': 'HR',
        'human resources': 'HR',
        'Human Resources': 'HR',
        'finance': 'Finance',
        'FINANCE': 'Finance',
        'it': 'IT',
        'IT': 'IT'
    }

    df_cleaned['department'] = (df_cleaned['department']
                                .replace(dept_mapping)
                                .replace({'': np.nan, 'Unknown': np.nan}))

    print(f"Standardized departments:")
    print(df_cleaned['department'].value_counts())

    return df_cleaned

def clean_numeric_data(self, df):
    """Clean numeric columns (age, salary, etc.)"""

    df_cleaned = df.copy()

    # Clean age column
    if 'age' in df_cleaned.columns:
        print("\n=== CLEANING AGE DATA ===")

        original_age = df_cleaned['age'].copy()

        # Convert to numeric, coercing errors to NaN
        df_cleaned['age'] = pd.to_numeric(df_cleaned['age'], errors='coerce')

        # Set reasonable bounds for age
        df_cleaned.loc[(df_cleaned['age'] < 18) | (df_cleaned['age'] > 100), 'age'] = np.nan

        # Fill missing ages with median
        median_age = df_cleaned['age'].median()
        df_cleaned['age'].fillna(median_age, inplace=True)

        print(f"Age statistics after cleaning:")
        print(df_cleaned['age'].describe())

    # Clean salary column
    if 'salary' in df_cleaned.columns:
        print("\n=== CLEANING SALARY DATA ===")

        # Convert to numeric
        df_cleaned['salary'] = pd.to_numeric(df_cleaned['salary'], errors='coerce')

        # Set reasonable bounds for salary (assuming USD)

```

```

df_cleaned.loc[(df_cleaned['salary'] < 20000) |
               (df_cleaned['salary'] > 1000000), 'salary'] = np.nan

# Fill missing salaries based on department median
if 'department' in df_cleaned.columns:
    df_cleaned['salary'] = (df_cleaned.groupby('department')['salary']
                           .transform(lambda x: x.fillna(x.median()))))

# Fill any remaining missing values with overall median
df_cleaned['salary'].fillna(df_cleaned['salary'].median(), inplace=True)

print(f"Salary statistics after cleaning:")
print(df_cleaned['salary'].describe())

return df_cleaned

def clean_date_data(self, df):
    """Clean date columns"""

    df_cleaned = df.copy()

    if 'join_date' in df_cleaned.columns:
        print("\n=== CLEANING DATE DATA ===")

        # Try multiple date formats
        date_formats = [
            '%Y-%m-%d',      # 2023-01-15
            '%m/%d/%Y',      # 01/15/2023
            '%d-%b-%Y',      # 15-Jan-2023
            '%Y/%m/%d',      # 2023/01/15
            '%B %d, %Y',     # January 15, 2023
            '%d/%m/%Y'       # 15/01/2023
        ]

        df_cleaned['join_date_cleaned'] = pd.NaT

        for fmt in date_formats:
            mask = df_cleaned['join_date_cleaned'].isna()
            try:
                df_cleaned.loc[mask, 'join_date_cleaned'] = pd.to_datetime(
                    df_cleaned.loc[mask, 'join_date'],
                    format=fmt,
                    errors='ignore'
                )
            except:
                continue

        # Set reasonable date bounds (e.g., between 2000 and current date)
        current_date = datetime.now()
        min_date = datetime(2000, 1, 1)

        date_mask = ((df_cleaned['join_date_cleaned'] < min_date) |
                     (df_cleaned['join_date_cleaned'] > current_date))
        df_cleaned.loc[date_mask, 'join_date_cleaned'] = pd.NaT

        print(f"Date parsing results:")

```

```

        print(f"Successfully parsed: {df_cleaned['join_date_cleaned'].notna().sum()}")
        print(f"Failed to parse: {df_cleaned['join_date_cleaned'].isna().sum()}")
        print(f>Date range: {df_cleaned['join_date_cleaned'].min()} to {df_cleaned['join_date_cleaned'].max()}")

    return df_cleaned

def handle_duplicates(self, df):
    """Handle duplicate records"""

    print("\n=== HANDLING DUPLICATES ===")

    # Identify duplicates
    duplicate_mask = df.duplicated()
    duplicate_count = duplicate_mask.sum()

    print(f"Found {duplicate_count} duplicate rows")

    if duplicate_count > 0:
        # Show some duplicate examples
        print("Sample duplicate rows:")
        duplicates = df[duplicate_mask].head()
        print(duplicates)

        # Remove duplicates, keeping first occurrence
        df_deduped = df.drop_duplicates(keep='first')

        print(f"Shape after deduplication: {df_deduped.shape}")
        return df_deduped

    return df

def _count_invalid_emails(self, email_series):
    """Count invalid email addresses"""
    if email_series.dtype != 'object':
        return 0

    email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    valid_emails = email_series.str.match(email_pattern, na=False)
    return (~valid_emails & email_series.notna()).sum()

def _count_invalid_phones(self, phone_series):
    """Count invalid phone numbers"""
    if phone_series.dtype != 'object':
        return 0

    # Simple phone validation (at least 10 digits)
    phone_digits = phone_series.str.extract(r'(\d+)')[0]
    valid_phones = phone_digits.str.len() >= 10
    return (~valid_phones & phone_series.notna()).sum()

def _standardize_phone_numbers(self, phone_series):
    """Standardize phone number format"""

    def clean_phone(phone):
        if pd.isna(phone) or phone == '':
            return np.nan

```

```

        # Extract only digits
        digits = re.sub(r'\D', '', str(phone))

        # Check if valid length
        if len(digits) == 10:
            # Format as (XXX) XXX-XXXX
            return f"({digits[:3]}) {digits[3:6]}-{digits[6:]}"
        elif len(digits) == 11 and digits.startswith('1'):
            # Remove country code and format
            digits = digits[1:]
            return f"({digits[:3]}) {digits[3:6]}-{digits[6:]}"
        else:
            return np.nan

    return phone_series.apply(clean_phone)

def generate_cleaning_summary(self, original_df, cleaned_df):
    """Generate comprehensive cleaning summary"""

    print("\n=== CLEANING SUMMARY ===")

    summary = {
        'original_shape': original_df.shape,
        'cleaned_shape': cleaned_df.shape,
        'rows_removed': original_df.shape[0] - cleaned_df.shape[0],
        'columns_added': cleaned_df.shape[1] - original_df.shape[1]
    }

    print(f"Original shape: {summary['original_shape']}")
    print(f"Cleaned shape: {summary['cleaned_shape']}")
    print(f"Rows removed: {summary['rows_removed']}")
    print(f"Columns added: {summary['columns_added']}")

    # Compare missing values
    print("\nMissing values comparison:")
    original_missing = original_df.isnull().sum()
    cleaned_missing = cleaned_df.isnull().sum()

    for col in original_df.columns:
        if col in cleaned_df.columns:
            orig_miss = original_missing[col]
            clean_miss = cleaned_missing[col]
            change = clean_miss - orig_miss
            print(f" {col}: {orig_miss} → {clean_miss} ({change:+d})")

    # Data quality improvement
    print("\nData quality improvements:")
    if hasattr(self, 'cleaning_report') and 'quality_issues' in self.cleaning_report:
        for col, issues in self.cleaning_report['quality_issues'].items():
            print(f" {col}: {len(issues)} issues addressed")

    return summary

# Comprehensive data cleaning demonstration
def run_data_cleaning_pipeline():

```

```

cleaner = DataCleaningPipeline()

# Step 1: Create dirty dataset
dirty_df = cleaner.create_dirty_dataset()

# Step 2: Assess data quality
quality_issues = cleaner.assess_data_quality(dirty_df)

# Step 3: Clean text data
df_text_cleaned = cleaner.clean_text_data(dirty_df)

# Step 4: Clean numeric data
df_numeric_cleaned = cleaner.clean_numeric_data(df_text_cleaned)

# Step 5: Clean date data
df_date_cleaned = cleaner.clean_date_data(df_numeric_cleaned)

# Step 6: Handle duplicates
df_final_cleaned = cleaner.handle_duplicates(df_date_cleaned)

# Step 7: Generate summary
summary = cleaner.generate_cleaning_summary(dirty_df, df_final_cleaned)

return dirty_df, df_final_cleaned, summary

# Run the complete pipeline
original_data, cleaned_data, cleaning_summary = run_data_cleaning_pipeline()

```

Quick Reference Guide

Essential Python Libraries for Data Science

```

essential_imports = {
    'data_manipulation': [
        'import pandas as pd',
        'import numpy as np'
    ],
    'visualization': [
        'import matplotlib.pyplot as plt',
        'import seaborn as sns',
        'import plotly.express as px'
    ],
    'machine_learning': [
        'from sklearn.model_selection import train_test_split',
        'from sklearn.preprocessing import StandardScaler',
        'from sklearn.ensemble import RandomForestClassifier',
        'from sklearn.metrics import accuracy_score, classification_report'
    ],
    'statistics': [
        'from scipy import stats',
        'from scipy.stats import ttest_ind, chi2_contingency'
    ]
}

```

```
]
}
```

Common Data Operations Cheat Sheet

```
# DataFrameOperations
df_operations = {
    'inspection': [
        'df.head()', 'df.tail()', 'df.info()', 'df.describe()',
        'df.shape', 'df.columns', 'df.dtypes', 'df.isnull().sum()'
    ],
    'selection': [
        'df["column"]', 'df[["col1", "col2"]]',
        'df.iloc[0:5]', 'df.loc[df["col"] > 5]',
        'df.query("col > 5 and col2 == \'value\')"
    ],
    'manipulation': [
        'df.drop(columns=["col"])', 'df.rename(columns={"old": "new"})',
        'df.sort_values("col")', 'df.groupby("col").agg({"col2": "mean"})',
        'pd.concat([df1, df2])', 'pd.merge(df1, df2, on="key")'
    ],
    'missing_data': [
        'df.dropna()', 'df.fillna(value)', 'df.ffill()', 'df.bfill()',
        'df.interpolate()', 'df.replace(to_replace, value)'
    ]
}
```

Performance Optimization Tips

1. **Use vectorized operations** instead of loops
2. **Choose appropriate data types** (int32 vs int64, category vs object)
3. **Use chunking** for large datasets that don't fit in memory
4. **Leverage NumPy** for numerical computations
5. **Use pandas methods** instead of pure Python
6. **Profile your code** to identify bottlenecks
7. **Consider using Dask** for parallel computing

Common Interview Patterns

```
interview_patterns = {
    'data_loading': 'How do you read different file formats?',
    'data_cleaning': 'How do you handle missing values and outliers?',
    'data_transformation': 'How do you reshape and pivot data?',
    'data_aggregation': 'How do you perform groupby operations?',
    'data_merging': 'How do you join multiple datasets?',
    'data_visualization': 'How do you create effective plots?',
    'performance': 'How do you optimize pandas operations?'
}
```

This comprehensive Python for Data Science cheat sheet covers essential concepts, libraries, and operations needed for data science interviews. Master these fundamentals and practice implementing complete data science workflows!